COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 20, 2026	**DATE SUBMITTED:**	October 20, 2026
NAME:	Payas, John Paul S. Reyes, Fenzie Aaron A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HANDS-ON ACTIVITY NO. 14.1 **ALGORITHMIC COMPLEXITY**

1. PROCEDURE OUTPUT

	Best Case	Worst Case	Analysis
Algorithm 1: Bubble Sort	O(n)	$O(n^2)$	The best case occurs when the array is already sorted and an optimized Bubble Sort detects no swaps. The worst case occurs with reverse-sorted data, requiring many comparisons and swaps.
Algorithm 2: Selection Sort	$O(n^2)$	$O(n^2)$	The algorithm searches for the smallest element in the unsorted portion during every pass. Even if the array is already sorted, it still performs comparisons, resulting in $O(n^2)$ time complexity in both cases.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

Reference – Hands-On Activity 7.1 (sorting.h file and main.cpp file)

	Algorithm 1	Algorithm 2

Best Case	**100 as Input Size**
	`TABLE 14-2: EXPERIMENTAL ANALYSIS`
	
	`Input Size: 100`
	`Best Case:`
	`Bubble Sort: 0.0002 ms`
	
	1000 as Input Size
	`Input Size: 1000`
	`Best Case:`
	`Bubble Sort: 0.0008 ms`
		100 as Input Size
		`TABLE 14-2: EXPERIMENTAL ANALYSIS`
		
		`Input Size: 100`
		`Best Case:`
		`Bubble Sort: 0.0002 ms`
		`Selection Sort: 0.0048 ms`
		
		1000 as Input Size
		`Input Size: 1000`
		`Best Case:`
		`Bubble Sort: 0.0008 ms`
		`Selection Sort: 0.3844 ms`

	10000 as Input Size Input Size: 10000 Best Case: Bubble Sort: 0.0075 ms	**10000 as Input Size** Input Size: 10000 Best Case: Bubble Sort: 0.0075 ms Selection Sort: 33.5221 ms
Worst Case	**100 as Input Size** Worst Case: Bubble Sort: 0.0156 ms	**100 as Input Size** Worst Case: Bubble Sort: 0.0156 ms Selection Sort: 0.0057 ms
	1000 as Input Size Worst Case: Bubble Sort: 1.4639 ms	**1000 as Input Size** Worst Case: Bubble Sort: 1.4639 ms Selection Sort: 0.3702 ms
	10000 as Input Size Worst Case: Bubble Sort: 142.3058 ms	**10000 as Input Size** Worst Case: Bubble Sort: 142.3058 ms Selection Sort: 33.4530 ms

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

Hands-On Activity 7.1 (Revision of Code in Accordance to the Previous)

(sorting.h file)

```cpp
// BUBBLE SORTING
template <typename T>
void bubbleSort(T arr[], size_t maxSize) {
    for (size_t i = 0; i < maxSize - 1; i++) {
        bool swapped = false;

        for (size_t j = 0; j < maxSize - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }

        if (!swapped)
            break;
    }
}

```

```cpp
// SELECTION SORTING
template <typename T>
void selectionSort(T arr[], int maxSize) {
    int minVal;

    for (int i = 0; i < maxSize - 1; i++) {
        minVal = routine_smallest(arr, i, maxSize);
        std::swap(arr[i], arr[minVal]);
    }
}

```

(main.cpp)

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <chrono>
#include <iomanip>
#include "sorting.h"

// MEASURE BUBBLE SORT
double measureBubble(const std::vector<int>& values) {
    std::vector<int> arr = values;

    auto start = std::chrono::high_resolution_clock::now();
    bubbleSort(arr.data(), arr.size());
    auto end = std::chrono::high_resolution_clock::now();

    return std::chrono::duration<double, std::milli>(end - start).count();
}

```

```cpp
// MEASURE SELECTION SORT
double measureSelection(const std::vector<int>& values) {
    std::vector<int> arr = values;

    auto start = std::chrono::high_resolution_clock::now();
    selectionSort(arr.data(), static_cast<int>(arr.size()));
    auto end = std::chrono::high_resolution_clock::now();

    return std::chrono::duration<double, std::milli>(end - start).count();
}

int main() {
    int sizes[] = {100, 1000, 10000};

    std::cout << std::fixed << std::setprecision(4);
    std::cout << "TABLE 14-2: EXPERIMENTAL ANALYSIS\n";

    for (int n : sizes) {
        std::vector<int> sorted(n);
        std::vector<int> reversed(n);

        for (int i = 0; i < n; i++) {
            sorted[i] = i + 1;
            reversed[i] = n - i;
        }

        std::cout << "\nInput Size: " << n << "\n";

        std::cout << "Best Case:\n";
        std::cout << "Bubble Sort: "
                  << measureBubble(sorted) << " ms\n";
        std::cout << "Selection Sort: "
                  << measureSelection(sorted) << " ms\n";

        std::cout << "Worst Case:\n";
        std::cout << "Bubble Sort: "
                  << measureBubble(reversed) << " ms\n";
        std::cout << "Selection Sort: "
                  << measureSelection(reversed) << " ms\n";
    }

    return 0;
}
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PSEUDOCODE BASED ON THE CODE

Binary Search Pseudocode:

```
BinarySearch(A, n, target)
    left = 0
    right = n - 1

    while left <= right
        mid = left + (right - left) / 2

        if A[mid] == target
            return mid
        else if A[mid] < target
            left = mid + 1
        else
            right = mid - 1

    return -1
```

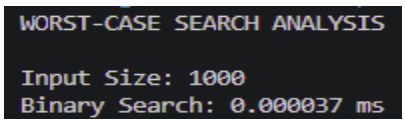
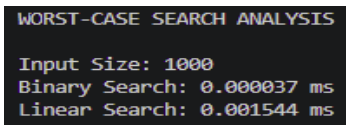
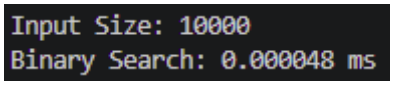
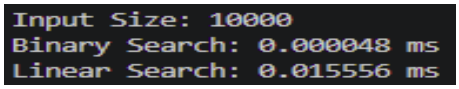
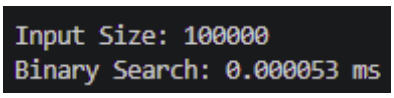
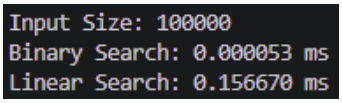
Linear Search Pseudocode:

```
LinearSearch(A, n, target)
    for i = 0 to n - 1
        if A[i] == target
            return i

    return -1
```

Time and Space Complexities

Analysis	Binary Search	Linear Search
Best-case time	O(1)	O(1)
Average-case time	O(log n)	O(n)
Worst-case time	O(log n)	O(n)
Auxiliary space (iterative)	O(1)	O(1)
Requirement	Sorted Array	Sorted array not required

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000		
10,000		
100,000		

ANALYSIS

Binary Search and Linear Search are two ways to find a value in an array. Through the use of Binary Search, it will work by dividing a sorted array in half each time, making it faster when going through large amounts of data. Its best-case time complexity is O(1), while its average and worst cases are O(log n). On the other hand, the Linear Search checks each element one by one and does not need a sorted array in order for it to work. Its best-case time complexity is O(1), while its average and worst cases are O(n). Both algorithms use O(1) extra space in their iterative versions. In the experiment that was altered coded using the previous code HOA7.1, both algorithms are tested with 1,000, 10,000, and 100,000 elements, using a value that is not in the array. As the number of elements increases, Linear Search is expected to take longer because it checks every element, while Binary Search needs fewer checks. Therefore we can say based on the result, Binary Search is better for large, sorted arrays, while Linear Search is simpler and can be used with unsorted arrays.

CONCLUSION – REYES

In this activity, we learned how to compare algorithms by studying their time and space complexities and measuring their execution times. Through the series of procedures and supplementary, we were able to see how Bubble Sort and Selection Sort have different best-case behaviors, but both have a worst-case time complexity of $O(n^2)$. Furthermore, I learned that Binary Search is faster for large, sorted arrays because it divides the search range in half, while Linear Search checks elements one by one which takes a lot of time in execution. By testing different input sizes, we can better understand how the number of elements affects an algorithm's performance. To sum these all up, this activity helped me understand the importance of choosing the right algorithm to make a program more efficient.

CONCLUSION – PAYAS

In this activity, we learned how to analyze and compare algorithms based on their time complexity and execution time. We understand that Bubble Sort and Selection Sort have different best-case behaviors but both have a worst-case time complexity of $O(n^2)$. We also learned that Binary Search is more efficient for large, sorted arrays because it divides the search range in half, while Linear Search checks each element one by one. By testing different input sizes, We gained a better understanding of how the number of elements affects an algorithm's performance.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT and Grammarly

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Error Debugging, Code Assistance (Alteration)

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: The AI only paved the way for me to realign or alter the previous work in my HOA7.1 to meet the required output in this HOA14.1, specifically in getting the best and worst case.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.